

# Relatório 5 - OpenMP

## 1. Enunciado

Implemente um programa em C que conte quantos números primos existem entre 2 e um valor máximo  $n$ . Depois, paralelize o laço principal usando a diretiva `#pragma omp parallel for` sem alterar a lógica original.

Compare o tempo de execução e os resultados das versões sequencial e paralela com diferentes valores para  $n$ .

Observe possíveis diferenças no resultado e no desempenho, e reflita sobre os desafios iniciais da programação paralela, como correção e distribuição de carga.

## 2. Objetivo da Tarefa

O objetivo desta atividade é investigar os desafios clássicos da paralelização de laços de repetição usando OpenMP:

- a **Condição de Corrida (Race Condition)**;
- e o **Desbalanceamento de Carga (Load Imbalance)**.

A tarefa visa demonstrar por que uma paralelização ingênua gera resultados incorretos na contagem de números primos e como resolver esses problemas utilizando as cláusulas `reduction` e `schedule(dynamic)`.

## 3. Metodologia

A solução foi desenvolvida na linguagem C utilizando a biblioteca OpenMP (`<omp.h>`). O programa calcula a quantidade de números primos no intervalo de 2 até  $n$  utilizando a função `is_prime(i)` (testando divisores até  $\sqrt{i}$ ).

Para comparar o comportamento e o desempenho, o código executa três abordagens sequencialmente dentro do mesmo `main`: 1. **Versão Sequencial**: Laço tradicional sem OpenMP para servir de linha de base (*baseline*). 2. **Versão Paralela Ingênua (`#pragma omp parallel for`)**: Aplicação direta da diretiva sem proteção na variável `contador`, evidenciando a ocorrência da condição de corrida. 3. **Versão Paralela Corrigida (`reduction(+:contador) schedule(dynamic)`)**: Utilização da redução para garantir a integridade da contagem e do escalonamento dinâmico para otimizar a distribuição do trabalho entre as threads.

Os tempos foram medidos com a função `omp_get_wtime()`. Os testes foram executados no macOS para diferentes valores de  $n$  (1.000.000, 5.000.000 e 10.000.000).

A compilação foi realizada via Clang: `clang -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include -L$(brew --prefix libomp)/lib -lomp tarefa5.c -o tarefa5`

## 4. Código-Fonte

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
// compilar: clang -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include -L$(brew --prefix libomp)
```

```
// Função que verifica se um número é primo
```

```
int is_prime(int num) {
    if (num <= 1) return 0;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return 0;
    }
    return 1;
}
```

```

}

int main() {
    int n = 5000000; // Altere este valor para testar (ex: 1000000, 5000000, 10000000)
    double inicio, fim;
    int contador;

    printf("Buscando primos de 2 ate %d...\n\n", n);

    // =====
    // 1. VERSÃO SEQUENCIAL
    // =====
    contador = 0;
    inicio = omp_get_wtime();
    for (int i = 2; i <= n; i++) {
        if (is_prime(i)) {
            contador++;
        }
    }
    fim = omp_get_wtime();
    printf("[Sequencial]\nResultado: %d primos\nTempo: %f segundos\n\n", contador, fim - inicio);

    // =====
    // 2. VERSÃO PARALELA (INGÊNUA - COM ERRO)
    // Apenas com o #pragma omp parallel for
    // =====
    contador = 0;
    inicio = omp_get_wtime();
    #pragma omp parallel for
    for (int i = 2; i <= n; i++) {
        if (is_prime(i)) {
            contador++; // CONDIÇÃO DE CORRIDA AQUI!
        }
    }
    fim = omp_get_wtime();
    printf("[Paralelo Ingenuo]\nResultado: %d primos (ERRO! Condição de corrida)\nTempo: %f segundos\n\n");

    // =====
    // 3. VERSÃO PARALELA CORRIGIDA
    // Com Reduction e Dynamic Scheduling
    // =====
    contador = 0;
    inicio = omp_get_wtime();
    #pragma omp parallel for reduction(+:contador) schedule(dynamic)
    for (int i = 2; i <= n; i++) {
        if (is_prime(i)) {
            contador++; // Seguro! O reduction cuida disso.
        }
    }
    fim = omp_get_wtime();
    printf("[Paralelo Corrigido]\nResultado: %d primos (Correto!)\nTempo: %f segundos\n\n", contador, fim - inicio);

    return 0;
}

```

## 5. Resultados

### 5.1. Tabela Comparativa de Tempos e Corretude ( $n = 5.000.000$ )

| Abordagem                 | Primos Encontrados | Corretude                | Tempo de Execução | Speedup vs Sequencial    |
|---------------------------|--------------------|--------------------------|-------------------|--------------------------|
| <b>Sequencial</b>         | 348.513            | <b>Correto</b>           | 1.397375 s        | 1.0x ( <i>Baseline</i> ) |
| <b>Paralelo Ingênuo</b>   | 337.939            | <b>Incorreto (Perda)</b> | 0.503252 s        | N/A (Resultado Inválido) |
| <b>Paralelo Corrigido</b> | 348.513            | <b>Correto</b>           | <b>0.319857 s</b> | <b>~4.37x</b>            |

### 5.2. O Desafio da Correção: Condição de Corrida com os Dados Reais

Na versão paralela ingênua (`#pragma omp parallel for`), múltiplas threads tentam executar a instrução `contador++` simultaneamente na mesma variável compartilhada.

Como a operação de incremento não é atômica no hardware (exige leitura, modificação e gravação na memória RAM), ocorreram colisões de escrita. Nos testes realizados para  $n = 5.000.000$ : \* A versão sequencial encontrou exatamente **348.513 primos**. \* A versão paralela ingênua contabilizou apenas **337.939 primos**, resultando na **perda exata de 10.574 incrementos** devido à sobreposição de escritas das threads.

A adição da cláusula `reduction(+:contador)` corrigiu completamente o problema, fornecendo uma cópia local privada da variável `contador` para cada thread e garantindo a contagem perfeita de 348.513 primos.

### 5.3. O Desafio do Desempenho: Balanceamento de Carga (*Load Imbalance*)

O teste de primalidade `is_prime(i)` **não possui tempo de execução constante**. Verificar se o número 2 é primo exige apenas 1 operação; verificar se um número ímpar grande próximo a 5.000.000 é primo exige milhares de divisões.

- **Escalonamento Estático Padrão (`schedule(static)`)**: Por padrão, o OpenMP divide o laço em blocos contínuos e iguais. As threads que recebem os números menores terminam o trabalho rapidamente e ficam ociosas, enquanto as threads com números maiores ficam sobrecarregadas.
- **Escalonamento Dinâmico (`schedule(dynamic)`)**: A cláusula distribui pequenas fatias de iterações sob demanda. À medida que uma thread conclui o seu lote, ela solicita um novo bloco de números.

**Análise de Desempenho:** A aplicação conjunta de `reduction` e `schedule(dynamic)` reduziu o tempo de execução de 1,397375 s (sequencial) para 0,319857 s, gerando um **Speedup real de aproximadamente 4,37x** ( $\frac{1.397375}{0.319857}$ ).

## 6. Conclusão

A atividade evidencia os dois pilares fundamentais da programação paralela em arquiteturas de memória compartilhada: **corretude** e **eficiência**. A simples inserção da diretiva `#pragma omp parallel for` sem análise de dependências invalida os resultados devido a condições de corrida na variável acumuladora. Além disso, problemas com custo computacional variável entre iterações exigem o uso do escalonamento dinâmico (`schedule(dynamic)`) para evitar a ociosidade de núcleos. Com a aplicação conjunta de `reduction` e `schedule(dynamic)`, alcançou-se uma solução matematicamente correta e com aceleração expressiva de desempenho.